Department of Computer Science and Engineering

Database Management Systems

Lab 07 | Triggers, Stored Procedures and Functions

Smart Hostel Management System – A Case Study

Objective

To understand and implement active and stored-program behaviour in MySQL — triggers, stored procedures and stored functions — on the Smart Hostel Management System database you designed in Lab 2 (ER Model) and implemented in Lab 3 (Relational Schema & DDL), so that the business rules of the hostel are enforced and executed inside the database rather than in an application.

Outcome

By completing this lab, students will learn to:

- Identify the situations in which a rule belongs inside the database, and choose between a trigger, a stored procedure and a stored function for a stated requirement.
- Create triggers with CREATE TRIGGER and choose the activation time and the event a stated requirement needs.
- Write a trigger body with FOR EACH ROW, BEGIN … END, conditional logic and the OLD and NEW row references.
- Reject an invalid modification from inside a trigger with SIGNAL SQLSTATE and a message of the required text.
- Record every change of a value in an audit relation, and invoke a stored procedure from inside a trigger.
- Create stored procedures with IN and OUT parameters and invoke them with CALL together with user-defined (@) variables.
- Declare local variables with DECLARE, assign them with SELECT … INTO, and control the flow of a routine with IF … END IF.
- Create stored functions with a declared RETURNS type and a data-access characteristic, and invoke them in a SELECT list.
- Change the statement terminator with DELIMITER, and make a script re-runnable with DROP … IF EXISTS.

Before You Begin

- Build the smart_hostel database by running Schema.sql and then Data.sql, both issued with this lab. Schema.sql is the Lab 3 DDL — the eleven relations and their constraints; Data.sql holds the Lab 3 sample rows and, at its end, four newly admitted students (PES23CS017 – PES23CS020) who have neither an allocation nor a payment. Select the database with USE smart_hostel; before you begin. Lab 07 adds one relation, PAYMENT_AUDIT, given in the Schema Reference; create it before Task 1. No other table is to be created and the Lab 3 schema is not to be redesigned.
- Before executing any command, set the prompt in the MySQL command line client so that your SRN and the active database are visible on every line: prompt (SRN@\U) [\d] >_ — with your own SRN in place of SRN. Every trigger and routine body contains semicolons. Change the statement terminator with DELIMITER $$ before each CREATE statement, close the body with END $$, and reset it with DELIMITER ; afterwards.
- If binary logging is enabled and log_bin_trust_function_creators is 0 (the MySQL 8.0 default), every CREATE FUNCTION must carry a characteristic clause; omitting it raises ERROR 1418. The functions of Task 3 only read data, so they are declared READS SQL DATA.
- Prefix every trigger with trg_, every procedure with sp_ and every function with fn_, and use the exact procedure and function names given in Tasks 2 and 3. Begin each object with DROP TRIGGER / DROP PROCEDURE / DROP FUNCTION IF EXISTS so that the script can be re-run.
- Execute the three tasks in the order given, on the same database. The triggers created in Task 1 remain in force during Tasks 2 and 3, so Tasks 2 and 3 work on the data Task 1 leaves behind.
- Execute every statement in MySQL and verify the result before recording the output. For each question, record the SQL you wrote together with a screenshot of its execution and its output — including the affected-row count or the error text, where these are what the question asks for. Your SRN must be clearly visible inside every screenshot.
- Write each statement with a comment giving its question number, for example -- Task 1.1, before the statement itself.

- Write your queries in notepad after executing in MySQL; you can save the notepad file as .sql for submission.

Deliverables

- A single PDF containing, in order, each of the 7 questions with the SQL you wrote and a screenshot of its output, plus the SQL file with the complete code which you implemented. Label each screenshot with its question number.
- File names:
  - <SRN>_Lab07.pdf
  - <SRN>_Lab07.sql
- Keep the database as it is after submission — the triggers of Task 1, the procedures and functions of Tasks 2 and 3, and the rows they leave behind are part of what is evaluated.

Tasks to be executed:

- Task 1: Triggers — Questions 1.1 – 1.3
- Task 2: Stored Procedures — Questions 2.1 – 2.2
- Task 3: Stored Functions — Questions 3.1 – 3.2
- 7 questions in total; every question is written on the Lab 3 schema and the data issued with this lab.

Schema Reference

The questions below refer only to the tables and columns listed here. Use these names exactly. The first eleven relations are the schema created by Lab 3; PAYMENT_AUDIT is the only relation Lab 07 adds.

Table	Columns	Primary key
HOSTEL	Hostel_ID, Hostel_Name, Location, Capacity	Hostel_ID
STAFF	Staff_ID, Name, Salary, Job, Hostel_ID, Supervisor_ID	Staff_ID
ROOM	Hostel_ID, Room_Number, Floor	(Hostel_ID, Room_Number)
BED	Hostel_ID, Room_Number, Bed_Number, Availability	(Hostel_ID, Room_Number, Bed_Number)
STUDENT	Student_ID, First_Name, Middle_Name, Last_Name, Gender, Date_of_Birth, Email_Address	Student_ID
STUDENT_PHONE	Student_ID, Phone_Number	(Student_ID, Phone_Number)

Table	Columns	Primary key
FACILITY	Facility_ID, Facility_Name, Operating_Hours, Hostel_ID	Facility_ID
USES	Student_ID, Facility_ID	(Student_ID, Facility_ID)
ALLOCATE	Hostel_ID, Room_Number, Student_ID, Allocation_Date, Check_in_Date, Check_out_Date	(Hostel_ID, Room_Number, Student_ID)
PAYMENT	Payment_ID, Amount, Payment_Type, Payment_Date, Student_ID	Payment_ID
RECEIPT	Receipt_ID, Receipt_Date, Payment_ID	Receipt_ID
PAYMENT_AUDIT	Audit_ID (auto increment), Payment_ID, Student_ID, Old_Amount, New_Amount, Action_Type, Changed_On	Audit_ID

Identifiers used in the issued data: H01–H04, ST01–ST08, PES23CS001–PES23CS020, F01–F06, PAY001–PAY016, RCP001–RCP016. Availability holds 'Y' or 'N'. In PAYMENT_AUDIT, Action_Type holds 'UPDATE' or 'DELETE', Old_Amount and New_Amount are DECIMAL(10,2) and may be null, and Changed_On is a DATETIME.

Note on Triggers and Stored Routines

A program stored in the database is a trigger, a procedure or a function. The three are not interchangeable, and the table below is the basis on which the correct one is chosen in each task of this lab.

	Trigger	Stored Procedure	Stored Function
Created with	CREATE TRIGGER … BEFORE | AFTER INSERT | UPDATE | DELETE ON table	CREATE PROCEDURE	CREATE FUNCTION … RETURNS type
How it runs	Automatically, once for each row the triggering statement affects	Only by the CALL statement	Inside an expression — SELECT list, WHERE, another routine
Parameters	None; the row is reached through NEW and OLD	IN, OUT and INOUT	IN only — the mode is never written
Returns	Nothing; it may change NEW in a BEFORE trigger or refuse the statement with SIGNAL	No return value; results travel back through OUT / INOUT or as a result set	Exactly one scalar value through RETURN
Result set	Cannot produce one	Can produce one or more	Cannot produce one
COMMIT / ROLLBACK	Not permitted — runs inside the transaction of the triggering statement	Permitted	Not permitted
Characteristic clause	Not applicable	Not required	Required when binary logging is on and log_bin_trust_function_creators = 0

Task 1: Triggers

A trigger is a statement the system executes automatically as a side effect of a modification of the database. For each question below, create the trigger or triggers the stated behaviour requires, and demonstrate that behaviour with test statements of your own on the issued data. Record the trigger definition, the demonstration and its output.

1.1 Create a trigger on the PAYMENT table.

The trigger must allow a payment only if its amount is between 12000.00 and 55000.00 (12000.00 and 55000.00 are also allowed).

For any other amount, the payment must be stopped with the message 'Payment amount out of permitted range'. The old payments must not change.

To test, insert these five UPI payments for student PES23CS017 with today's date:

- PAY017 for 11999.99 — must be stopped
- PAY017 for 12000.00
- PAY018 for 55000.00
- PAY019 for 30000.00
- PAY020 for 55000.01 — must be stopped

Then display PAY017, PAY018 and PAY019, the total number of rows in PAYMENT, and PAY002 to prove that it did not change.

1.2 Create two triggers on the PAYMENT table.

The first trigger applies the amount rule of 1.1 when an amount is changed: a new amount below 12000.00 or above 55000.00 must be stopped with the message 'Payment amount out of permitted range'.

The second trigger saves one row in PAYMENT_AUDIT every time an amount really changes. The row holds the payment ID, the student ID, the old amount, the new amount and the word 'UPDATE'. If the amount does not change, nothing is saved.

To test, make these three changes:

- Change the amount of PAY018 to 60000.00 — must be stopped.
- Change the amount of PAY019 to 40000.00 — must be saved in PAYMENT_AUDIT.
- Change the Payment_Type of PAY019 to 'CARD' — nothing is saved.

Then display PAYMENT_AUDIT. It must have only one row: PAY019, old amount 30000.00, new amount 40000.00.

1.3 This question has two parts: a stored procedure, and a trigger that invokes the procedure.
Part 1: Create a stored procedure sp_issue_receipt. It takes one payment ID and adds one receipt for that payment to the RECEIPT table. The receipt ID is 'RCP' followed by the number in the payment ID (for PAY022 the receipt ID is RCP022), and the receipt date is today.
Part 2: Create a trigger on the PAYMENT table that invokes sp_issue_receipt every time a new payment is added, so that every new payment gets its receipt automatically.
To test, insert a UPI payment PAY022 of 20000.00 for student PES23CS017 with today's date. Do not insert anything into RECEIPT yourself.
Then display the receipt of PAY022: it must be RCP022 with today's date.

Task 2: Stored Procedures

A stored procedure carries out a task rather than returning a value. It may take parameters in three modes, return one or more result sets, change the data, and hold a whole business rule — validation, decision and update — as one unit that an application calls by name. No stored function exists yet while Task 2 is executed, so every value a procedure needs is computed inside the procedure itself. The triggers of Task 1 remain in force.

2.1 Create a stored procedure sp_hostel_roster.
It takes one hostel ID and returns one row for every student who has a room in that hostel.
Each row has six columns:

- Room_Number
- Student_ID
- Student_Name — full name; no extra space if there is no middle name
- Gender
- Age — the age in full years
- Total_Paid — total amount paid, or 0.00 if nothing

Sort the rows by Room_Number, then by Student_ID.
Invoke the procedure for hostel H01 and display the result.

2.2 Create a stored procedure sp_payment_summary.

It takes one student ID and gives back two values to the caller: the total amount the student has paid (0.00 if nothing) and the number of payments the student has made.

Invoke it for student PES23CS004 and keep the two values in @total and @count. Display them in three columns: Student, Total_Paid and No_Of_Payments.

Do the same for student PES23CS007, who has not paid anything, so the result must be 0.00 and 0.

Task 3: Stored Functions

A stored function encapsulates a single derived value. The age of a student and the total a student has paid are values computed from the base tables rather than stored in them. Each is written once as a function and then used like any built-in function of MySQL, on the data Tasks 1 and 2 leave behind.

3.1 Create a stored function fn_student_age.

It takes one student ID and returns the age of that student in full years, as a whole number.

Display the age of student PES23CS001 in a column named Age_of_PES23CS001.

Then, using the function, display Student_ID, First_Name, Date_of_Birth and Age for the first 6 students, sorted by Student_ID.

3.2 Create a stored function fn_total_paid.

It takes one student ID and returns the total amount that student has paid. If the student has not paid anything, it must return 0.00, not NULL.

Display the totals of PES23CS001 and PES23CS007 next to each other, in two columns named Paid_001 and Paid_007.

Then display Student_ID and Total_Paid for the first 8 students, sorted by Student_ID.

Submission rules

- Both files are to be uploaded. A submission that is missing either of them is treated as incomplete.
- The prompt set at the start of the session must be visible in every screenshot. A screenshot in which your SRN cannot be read will not be evaluated.
- Screenshots are to be taken in the MySQL command line client, as in the earlier labs. Each one must display the statement together with what the server replied — the

returned rows, the affected-row count or the error text, as the question requires. A screenshot that displays only the text of a statement carries no marks.

- The SQL file must run from beginning to end on a database rebuilt from Schema.sql and Data.sql, without any error other than the errors the questions themselves require the server to raise. Begin each trigger, procedure and function with DROP … IF EXISTS so that this is possible.
- The names, parameter names, parameter modes, return types, message texts, status strings, identifiers and amounts given in the questions are part of the specification and are evaluated.
- Task 1 changes the data. Rebuild the database from Schema.sql and Data.sql if a question is to be executed again from the state it assumes.